CNN

2026.05.13

박철우

목차

1. MLP의 개념과 한계
2. CNN의 등장과 개념
3. CNN의 구조
4. ResNet
5. U-Net
6. CNN과 ViT의 차이

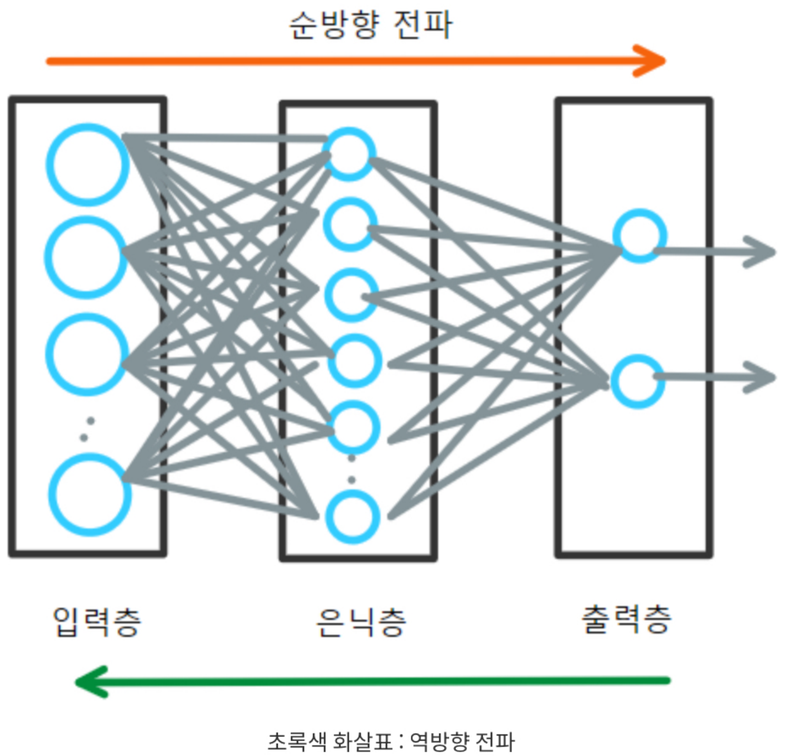
MLP의 개념과 한계

MLP란

여러 개의 퍼셉트론 뉴런을 여러 층으로 쌓은

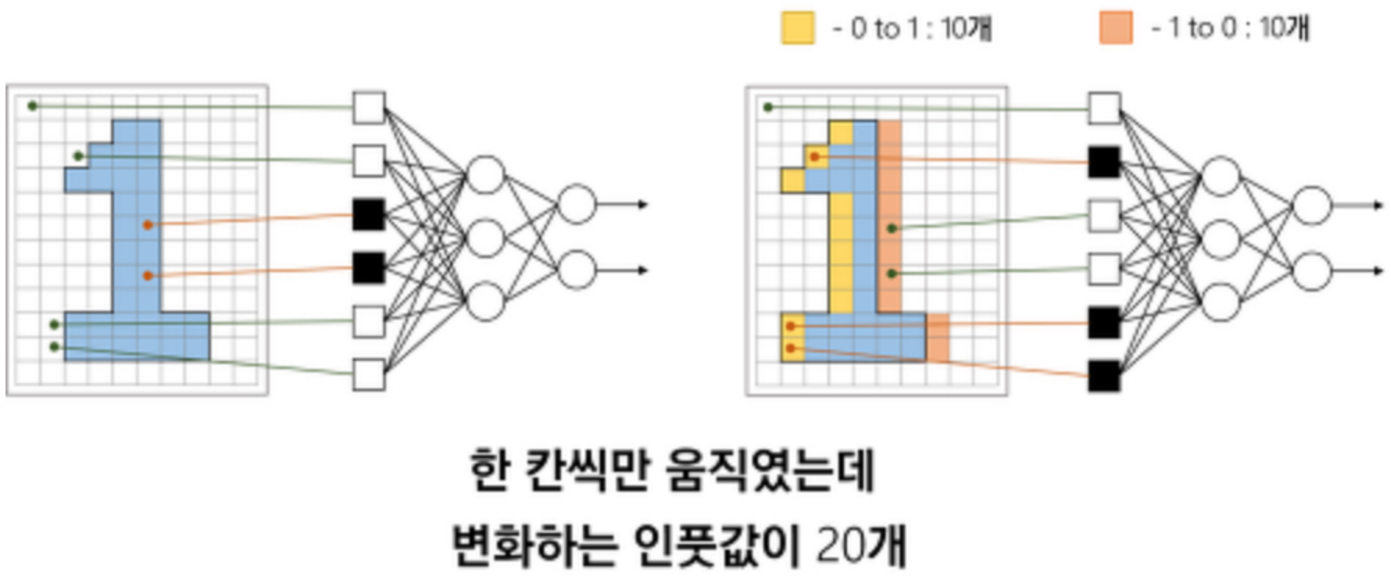
다층신경망 구조 입력층과 출력층 사이에 하나 이상의

은닉층을 가지고 있는 신경망



MLP의 한계

- 공간적/지역적 정보의 손실(이미지 처리 시)
- 비효율적인 파라미터 사용과 계산 복잡도
- 입력 변화에 대한 취약성
- 깊은 층에서의 학습 문제

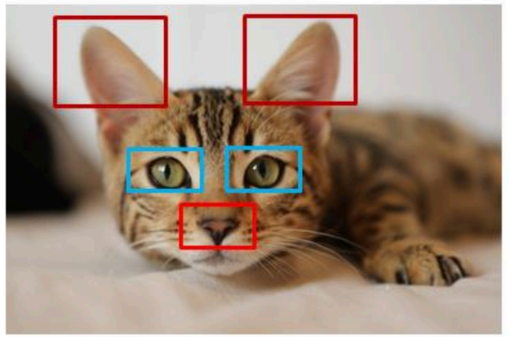


CNN의 등장

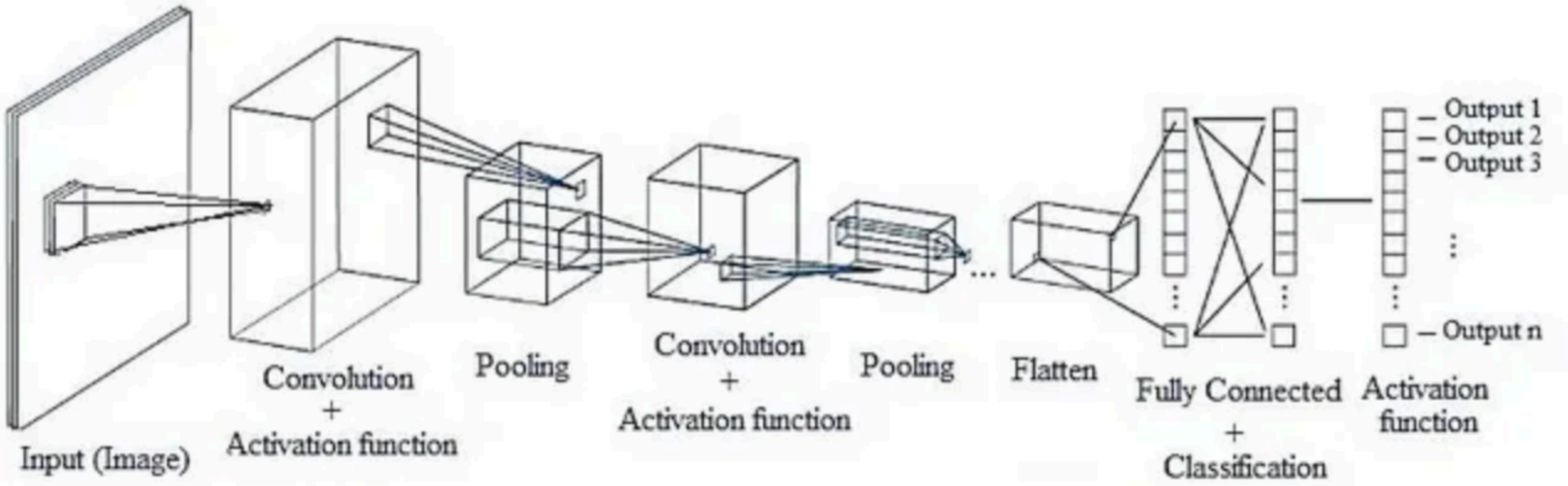
텍스트 1

MLP의 한계 극복

- 2차원 구조를 유지하여 공간 정보 활용
- 작은 필터를 여러 위치에 공유하여 파라미터 수 줄임
- Convolution 연산을 통해 작은 영역에서 특징 추출
- 작은 위치 변화에 강함



CNN의 구조


Input (Image)
Convolution
+
Activation function
Pooling
Convolution
+
Activation function
Pooling
...
Flatten
Fully Connected
+
Classification
Output 1
Output 2
Output 3
Output n
Activation
function

Input

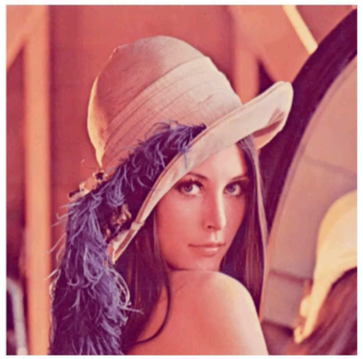
이미지 데이터 유형

GrayScale

· 가로 X 세로 X 1
1Channel

· 0 ~ 255 픽셀 값
(8bit)

Color



· 가로 X 세로 X 3
3Channel (R, G, B)

· 각 채널 0 ~ 255 픽셀 값
(8bit / 총 24bit)

Binary

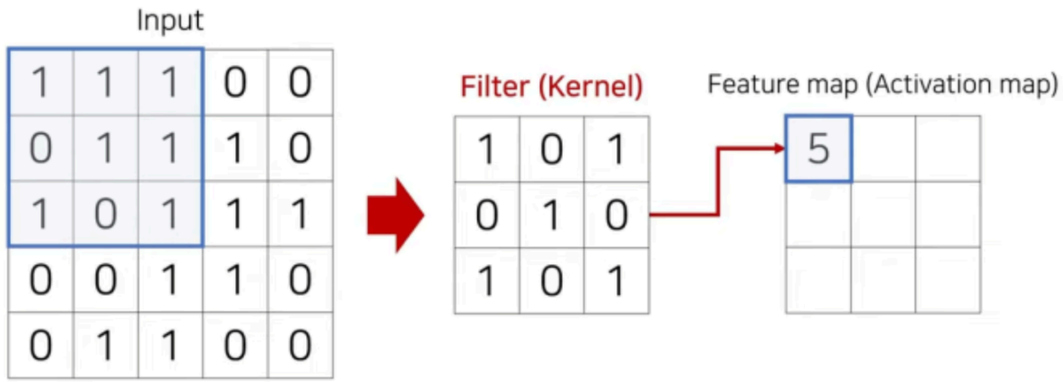
· 가로 X 세로

· 0 또는 1 픽셀 값
(1bit)

Convolution Layer

Filter and Kernel

- Kernel : 이미지의 작은 영역에 적용되는 학습 가능한 가중치 행렬
- Filter : 여러개의 Kernel을 합친 것
- 하나의 Filter는 하나의 Feature map 생성

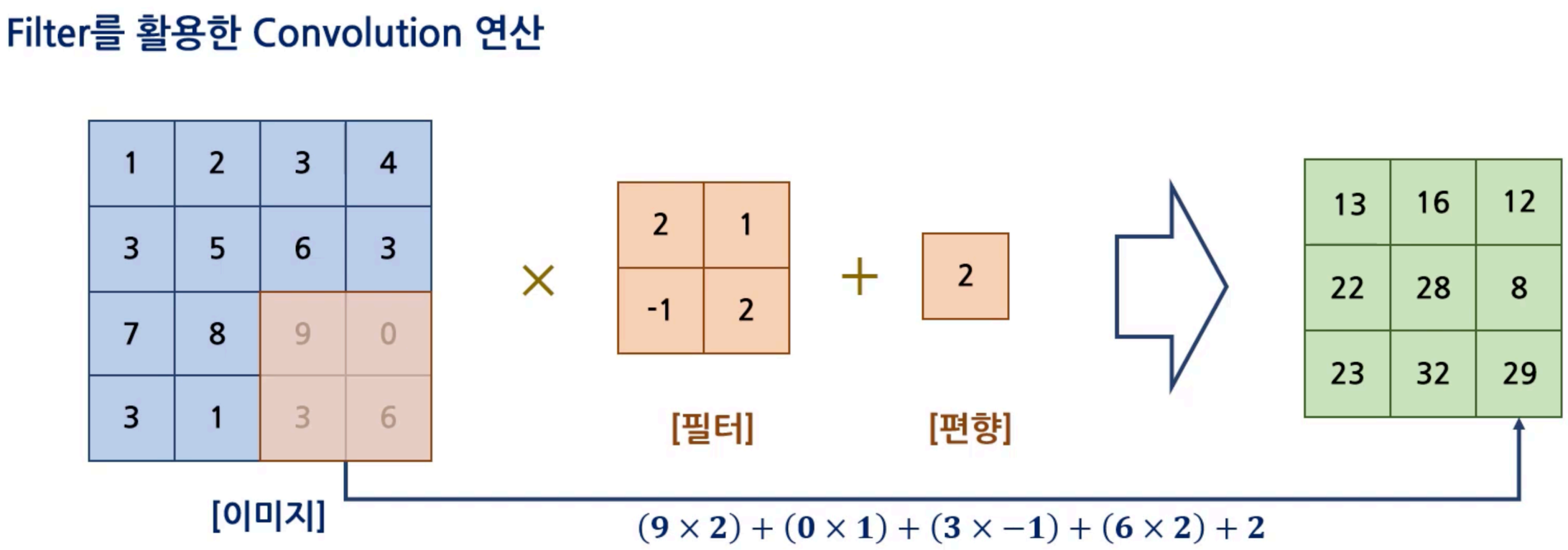


Feature map은 unit들이 같은 가중치

집합과 같은 bias 공유

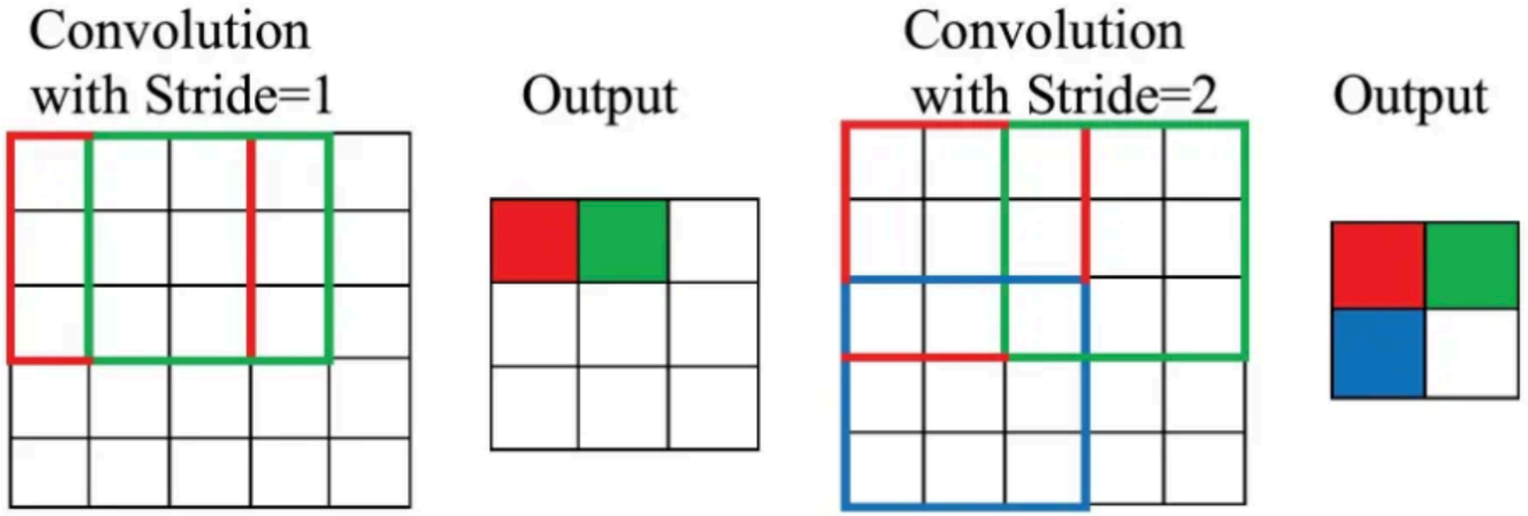
Convolution Layer

Convolution operation



Convolution Layer

Stride



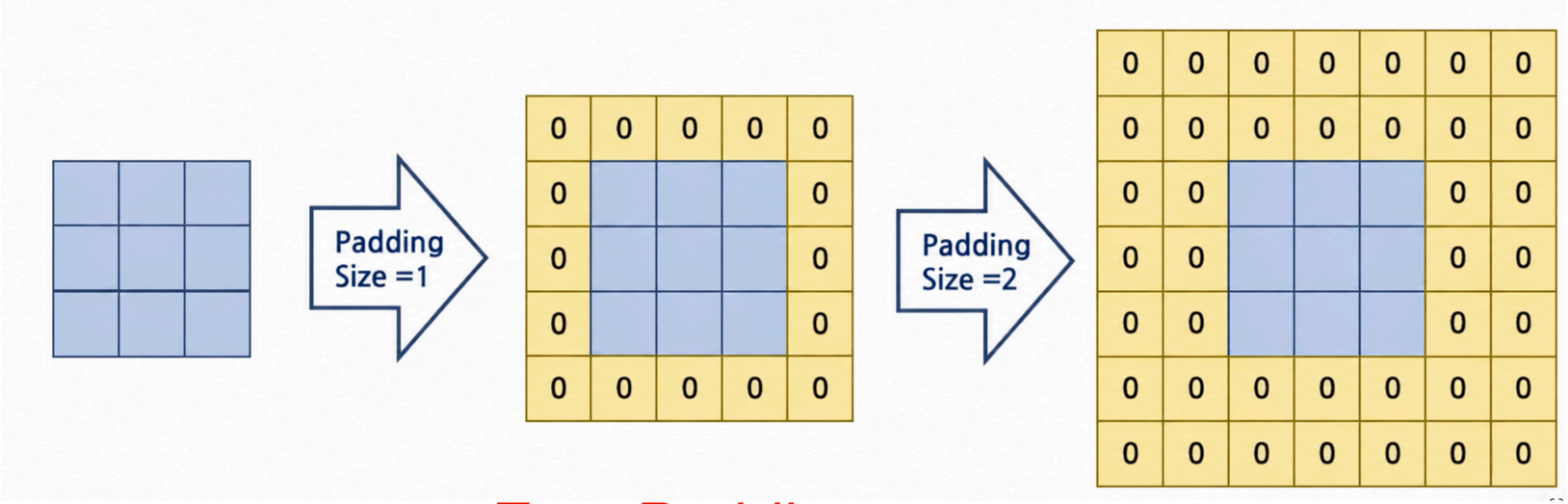
- **Convolution 연산을 여러번 수행하면 특성 맵이 작아지는 문제?**

- Filter/Kernel이 입력 이미지 위를 이동할 때, 한 번에 몇 칸씩 이동하는지를 정하는 값
- Stride가 커질수록 해상도↓, 연산량 ↓,출력 크기 ↓

Convolution Layer

Padding

- 입력 이미지의 가장자리에 값을 추가하는 방법



Zero Padding

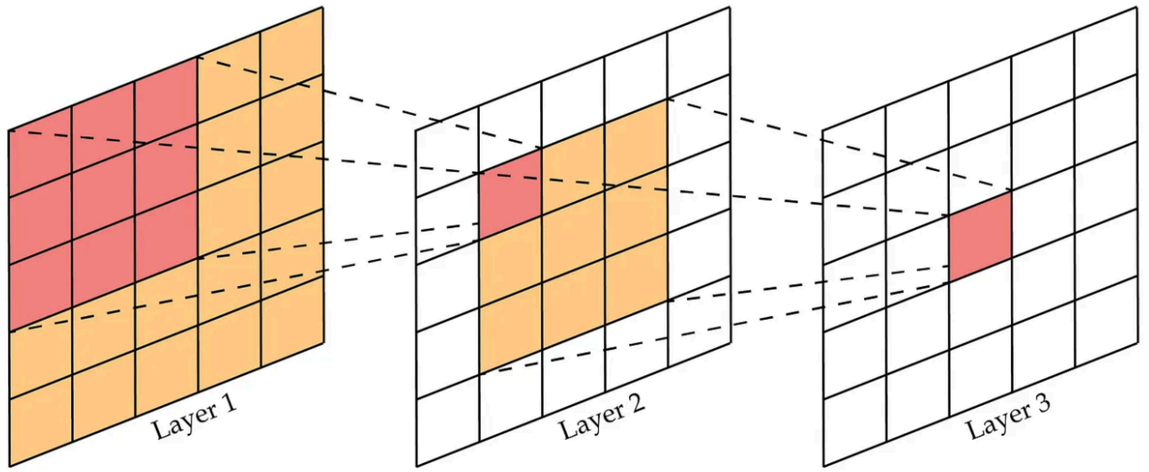
- 이미지 주변을 0으로 채우는 방법

출력 크기를 조정하고, 이미지의 가장자리 정보를 보존하기 위해 사용

Convolution Layer

Receptive Field

- 출력 레이어의 뉴런 하나에 영향을 미치는 입력 뉴런들의 공간크기



크기가 작을 때 : 작은 세부 특징(선,모서리 등)을 더 잘 감지, 빠른 연산, 경량화

크기가 클 때 : 얼굴 전체, 물체 형태 등 큰 특징 감지, 복잡한 패턴 인식, But 해상도 감소, 연산량 증가, 과적합 위험, Local 특징 간과 가능성

Kernel Size, Stride, Pooling, Layer Depth 등에 따라 크기가 달라짐

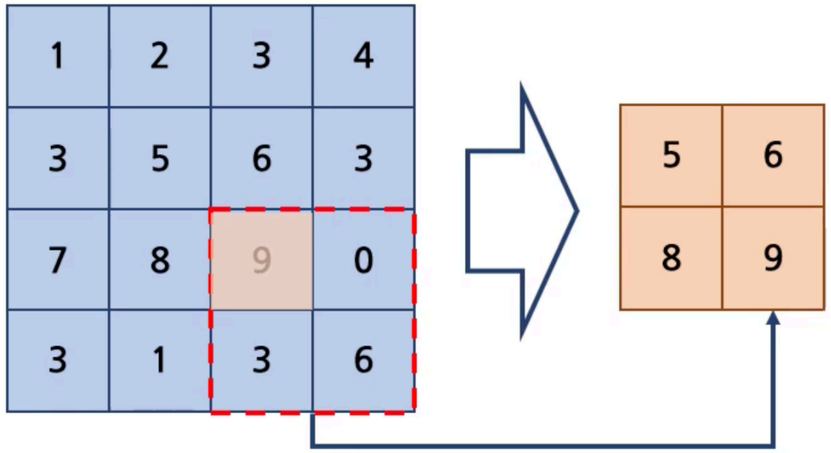
Pooling Layer

Feature Map의 작은 영역을 하나의 대표값으로 요약하는 층

Pooling Layer 종류

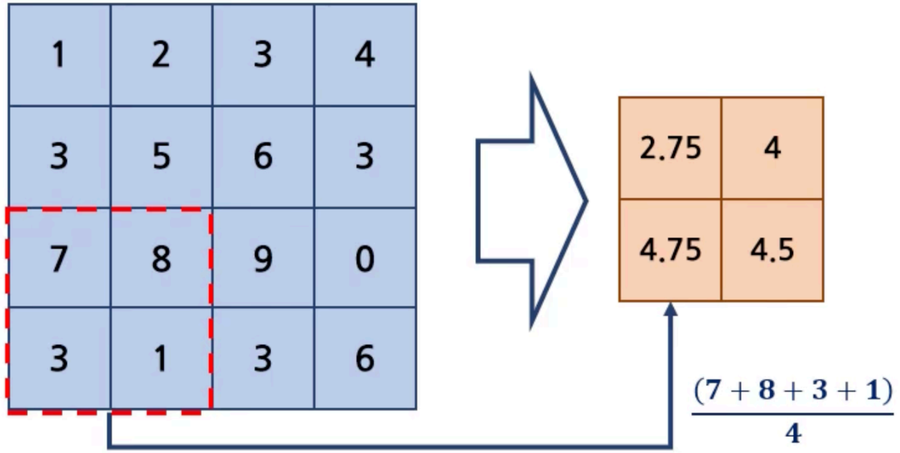
- MAX Pooling

Filter내에서 가장 큰 값을 추출



- Average Pooling

Filter내에서 평균 값을 추출

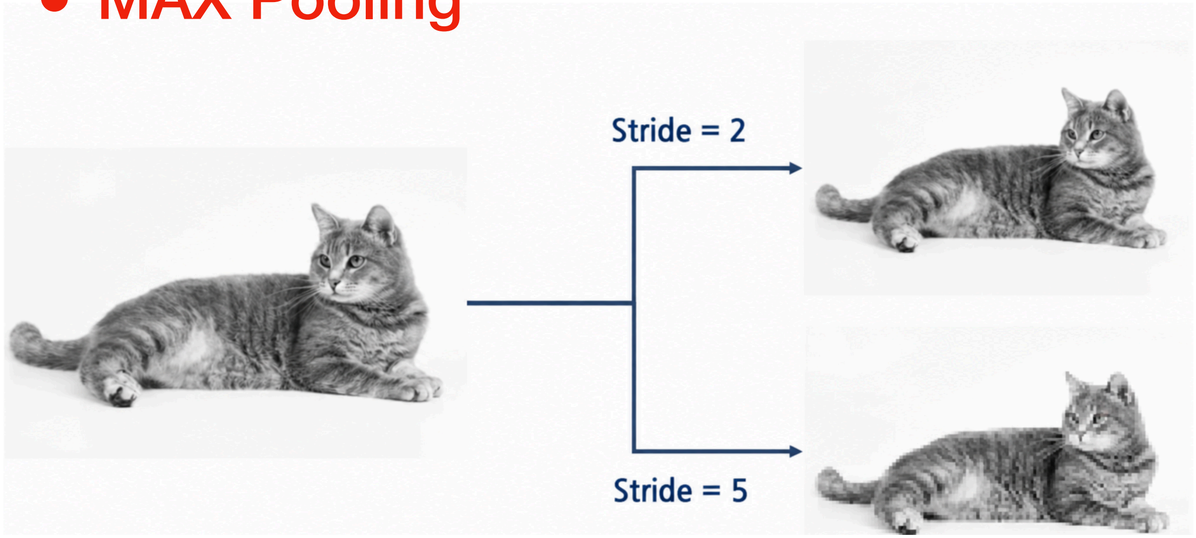


- 일반적으로 MAX Pooling이 Average Pooling 보다 좋음

Pooling Layer

Pooling layer층이 왜 중요한가?

- MAX Pooling



- 3x3 Filter 적용시

이미지 사이즈↓

화질↓

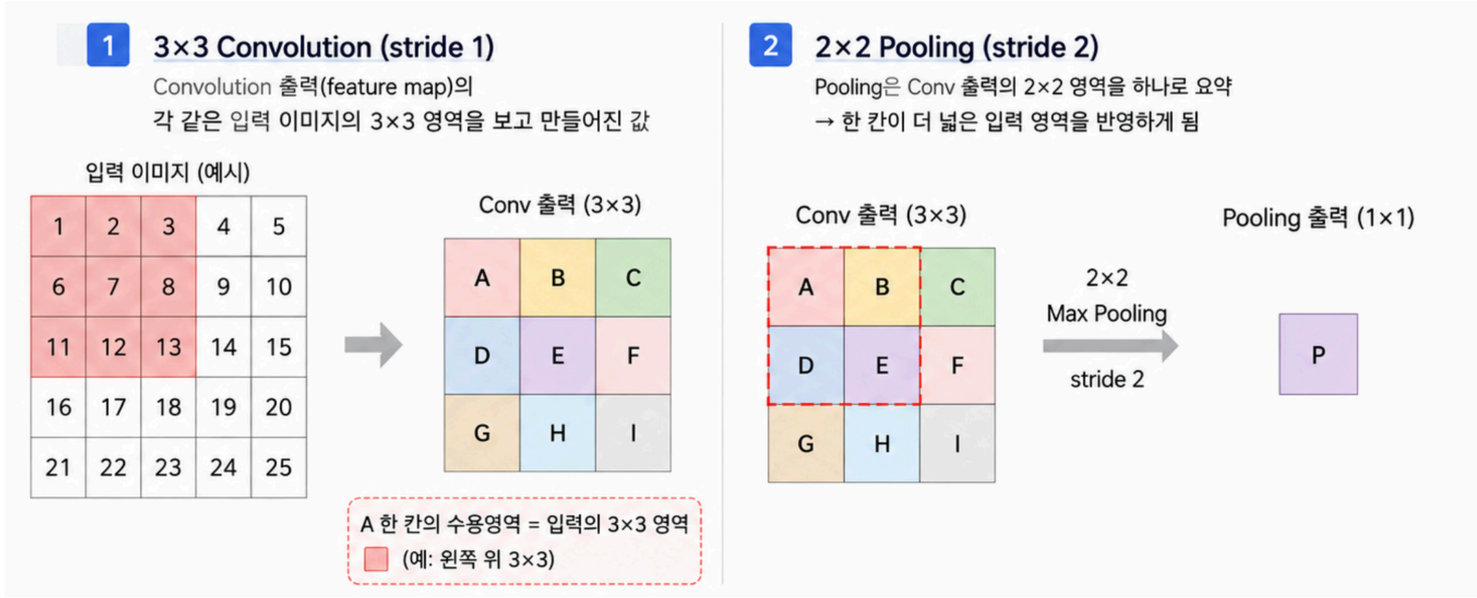
- 원본 이미지 그대로 학습 시킬 필요 없음
- 특징만 추출, 이미지 사이즈 줄이면 메모리 공간, 파라미터 수 감소, 성능 감소 적음

Pooling Layer

Pooling Layer와 Receptive Field

- Pooling을 하면 feature map의 크기는 줄어들지만, 층의 깊이가 깊어짐으로써 Receptive Field는 더 커진다.

- 즉, Pooling은 연산량을 줄이는 동시에 Receptive Field를 효과적으로 넓힘



CNN 기본 구조 코드

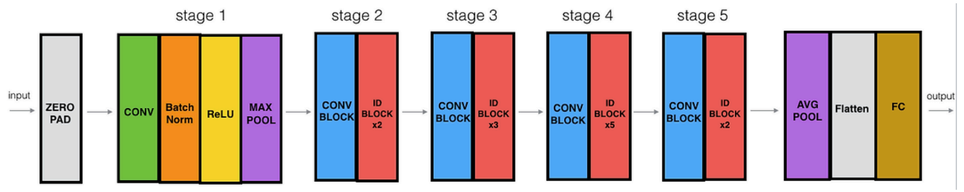
```python
import torch
import torch.nn as nn
import torch.nn.functional as F


class SimpleCNN(nn.Module):
    def __init__(self, num_classes=10):
        super(SimpleCNN, self).__init__()

        self.conv1 = nn.Conv2d(
            in_channels=3,        # 입력 채널 수: RGB 이미지라서 3
            out_channels=16,      # 3x3 필터 16개 사용 → feature map 16장 생성
            kernel_size=3,        # 3x3 크기의 필터
            stride=1,             # 필터가 한 칸씩 이동
            padding=1             # 가장자리에 0을 한 칸 추가하여 출력 크기 유지
        )

        self.pool1 = nn.MaxPool2d(
            kernel_size=2,        # 2x2 영역에서
            stride=2              # 두 칸씩 이동 → 높이/너비가 절반으로 감소
        )

        self.conv2 = nn.Conv2d(
            in_channels=16,       # 이전 conv1의 출력 채널 수 = 16
            out_channels=32,      # 필터 32개 사용 → feature map 32장 생성
            kernel_size=3,        # 3x3 크기의 필터
            stride=1,             # 한 칸씩 이동
            padding=1             # 출력 크기 유지
        )

        self.pool2 = nn.MaxPool2d(
            kernel_size=2,        # 2x2 영역에서 최댓값 선택
            stride=2              # 두 칸씩 이동 → 높이/너비가 다시 절반으로 감소
        )

        self.fc = nn.Linear(
            in_features=32 * 8 * 8,     # 32장의 8x8 feature map을 1차원으로 펼친 크기
            out_features=num_classes    # 최종 분류할 클래스 개수
        )
```

```python
    def forward(self, x):
        x = self.conv1(x)          # 첫 번째 convolution으로 지역적 특징 추출
        x = F.relu(x)              # ReLU 적용: 음수는 0, 양수는 그대로 유지
        x = self.pool1(x)          # max pooling으로 feature map 크기 감소

        x = self.conv2(x)          # 두 번째 convolution으로 더 복잡한 특징 추출
        x = F.relu(x)              # ReLU 적용
        x = self.pool2(x)          # max pooling으로 크기 다시 감소

        x = x.view(x.size(0), -1) # feature map을 1차원 벡터로 펼침, Flatten
        x = self.fc(x)             # fully connected layer로 최종 클래스 점수 출력

        return x
```

ResNet

ResNet의 등장

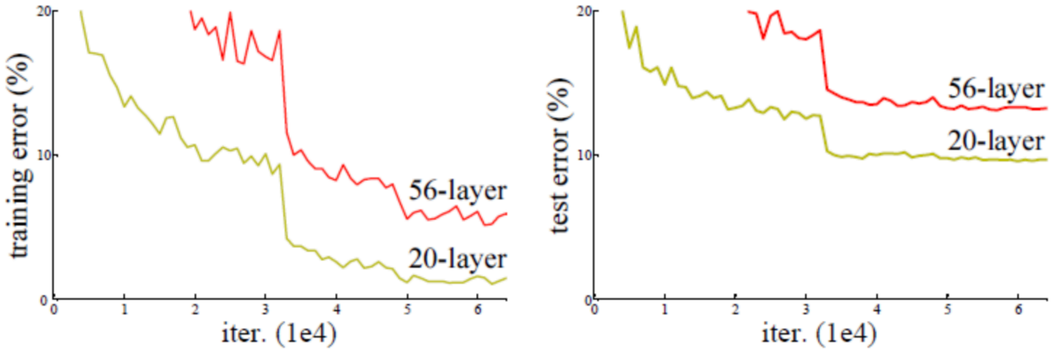
- LeNet -> AlexNet -> VGG/GoogLeNet-> ResNet



ResNet structure

왜 ResNet이 등장 하였을까?

-> CNN이 점점 깊어질수록 성능이 좋아진다고 생각 But, 너무 깊어지면 학습이 어려워지는 degradation 문제 발생, 이를 해결하기 위해 ResNet 등장

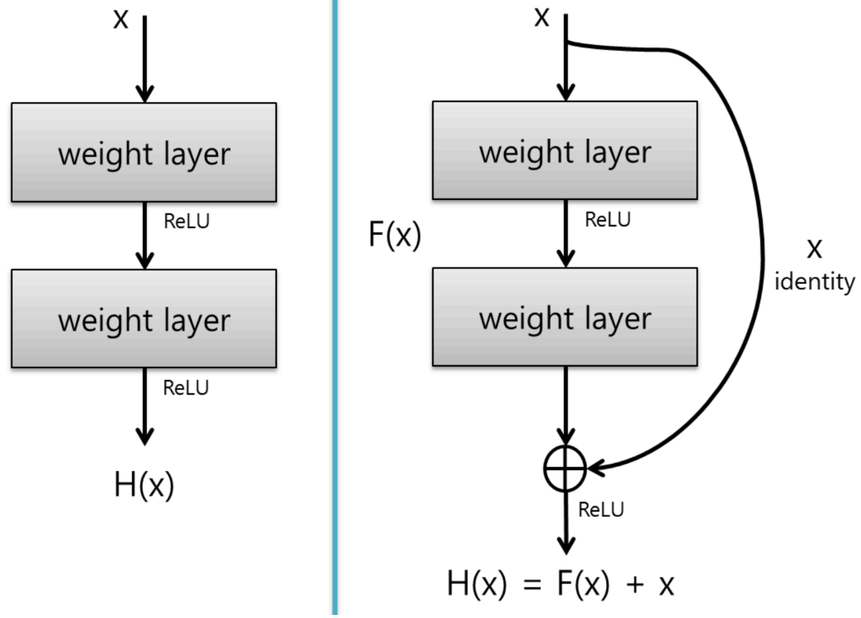


ResNet

Residual connection

- 기존 방식 : 여러 layer를 거쳐 H(x)를 직접 학습

- ResNet은 layer가 F(x)를 학습, 입력 x를 shortcut으로 전달해 F(x)에 더함 -> 직접 전달됨으로 네트워크 학습 쉬워짐



기존 방식 Residual Block

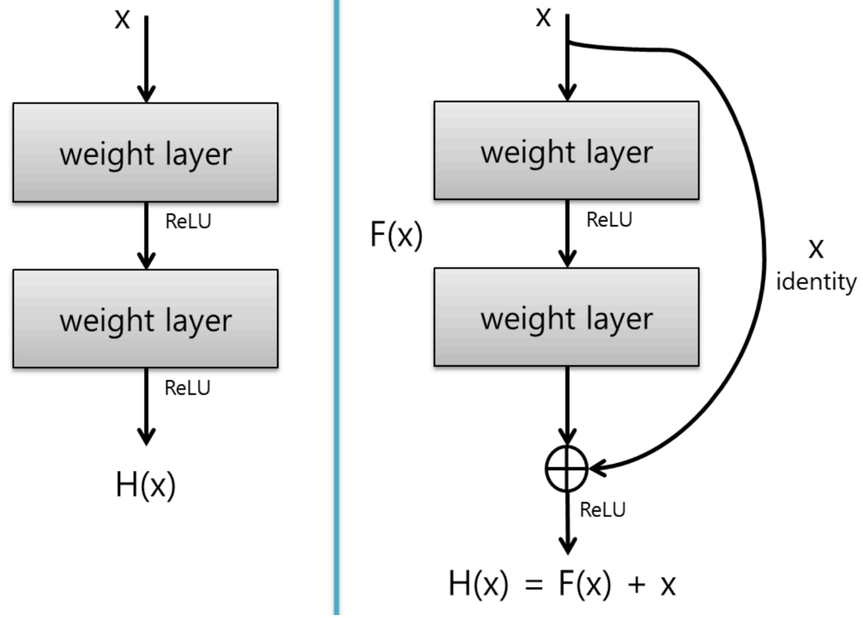
ResNet

Residual connection (Gradient Vanishing)

- Gradient Vanishing 란?

-> 신경망이 깊어질수록 앞쪽 layer로 전달 되는 gradient가 너무 작아져서, 앞쪽 layer가 거의 학습되지 않는 문제

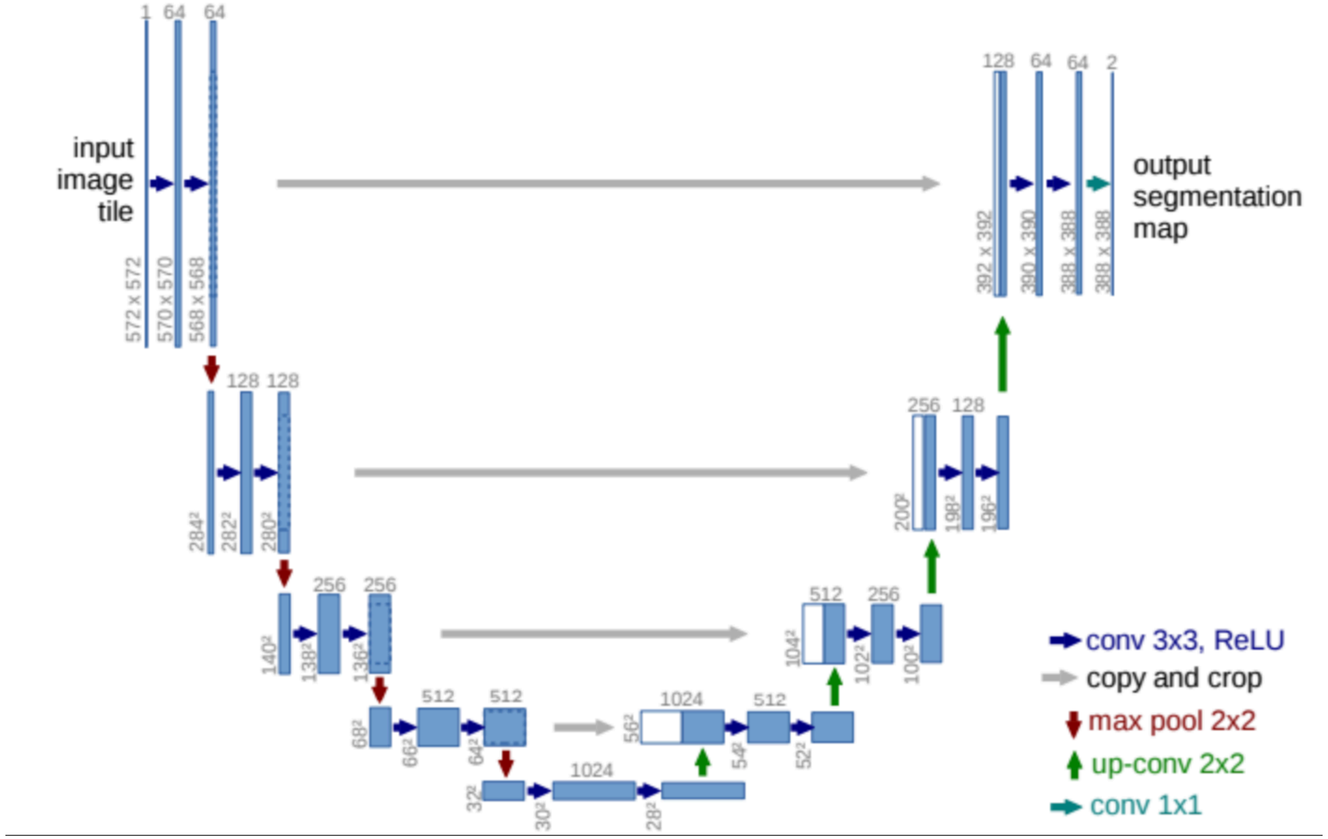
- Gradient Vanishing 문제 완화
  -> gradient가 shortcut 경로를 통해 직접 전달될 수 있기 때문



기존 방식 Residual Block

U-Net

U-Net의 구조

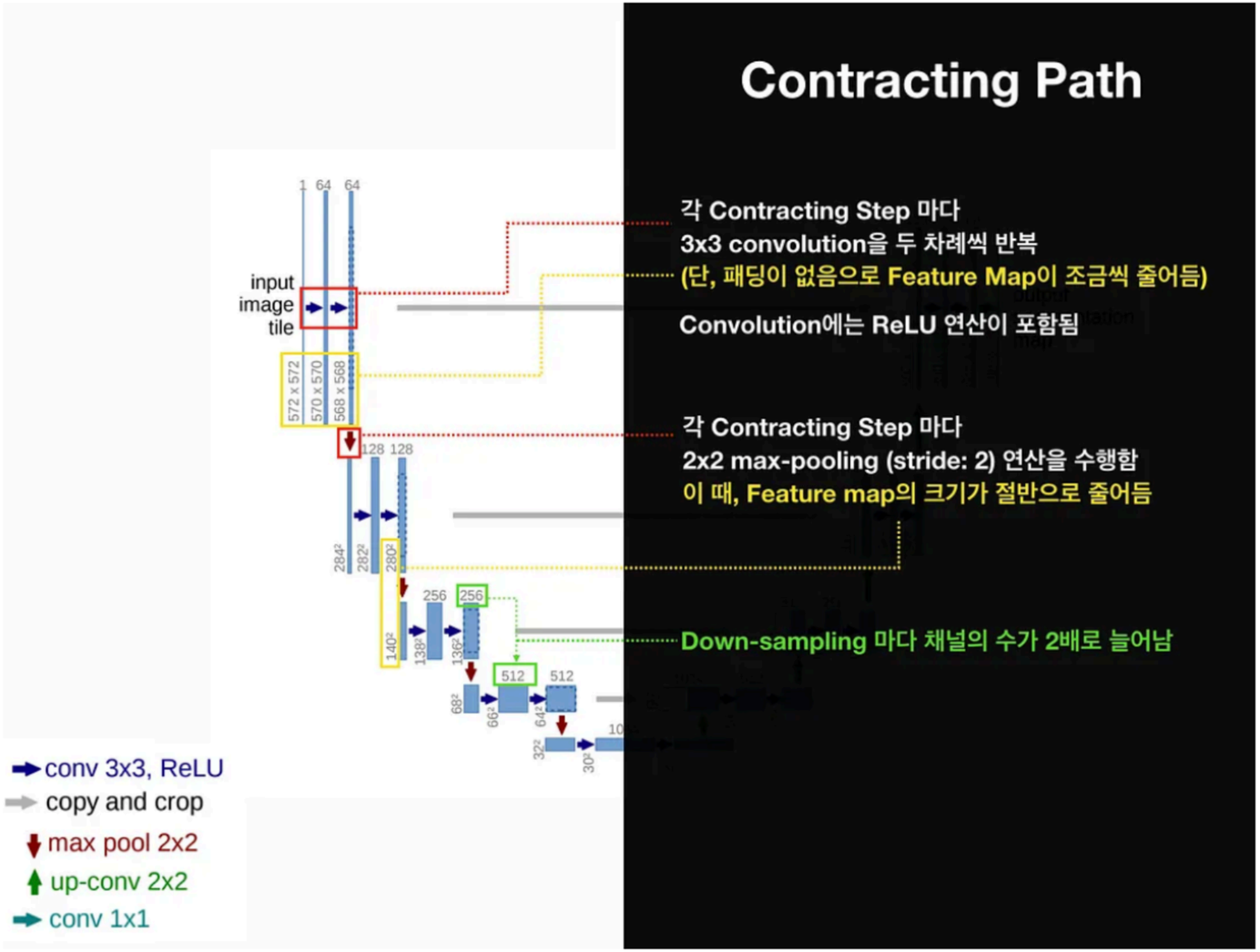


- Encoder -Decoder 구조

U-Net

Contracting Path

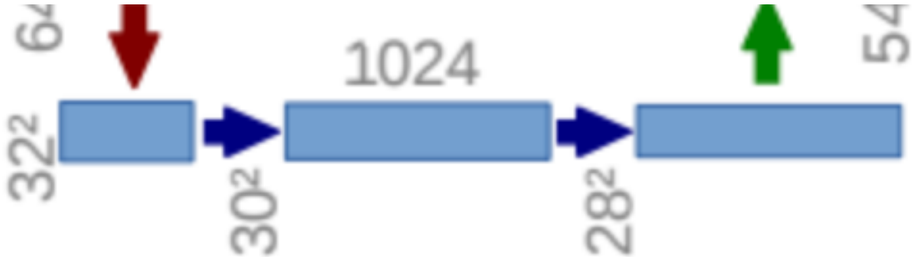
- 입력 이미지의 Context를 파악



U-Net

Bottleneck

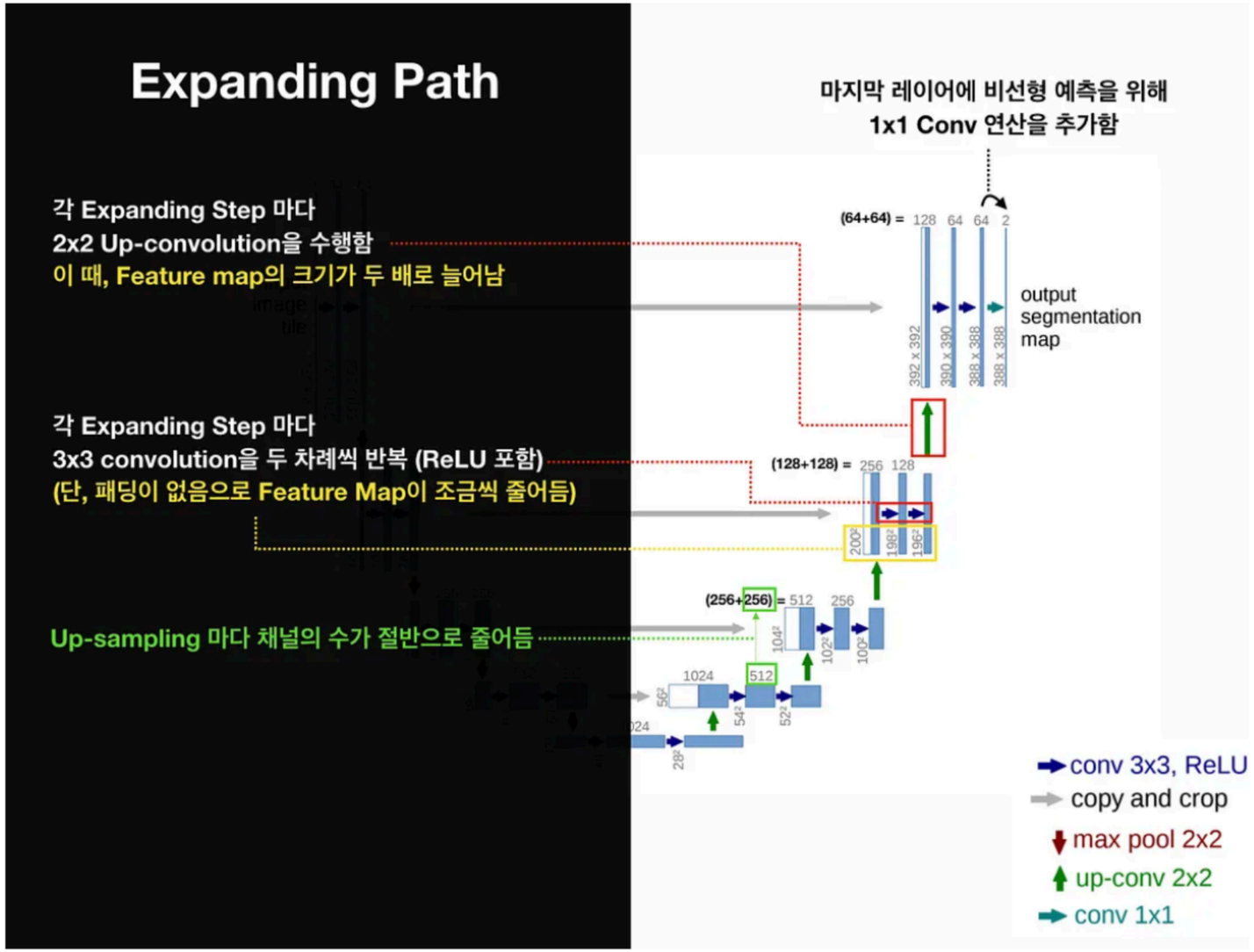
- 일종의 전환 구간
- 입력 이미지의 가장 추상적이고 고수준의 표현을 학습하는 부분



U-Net

Expanding Path

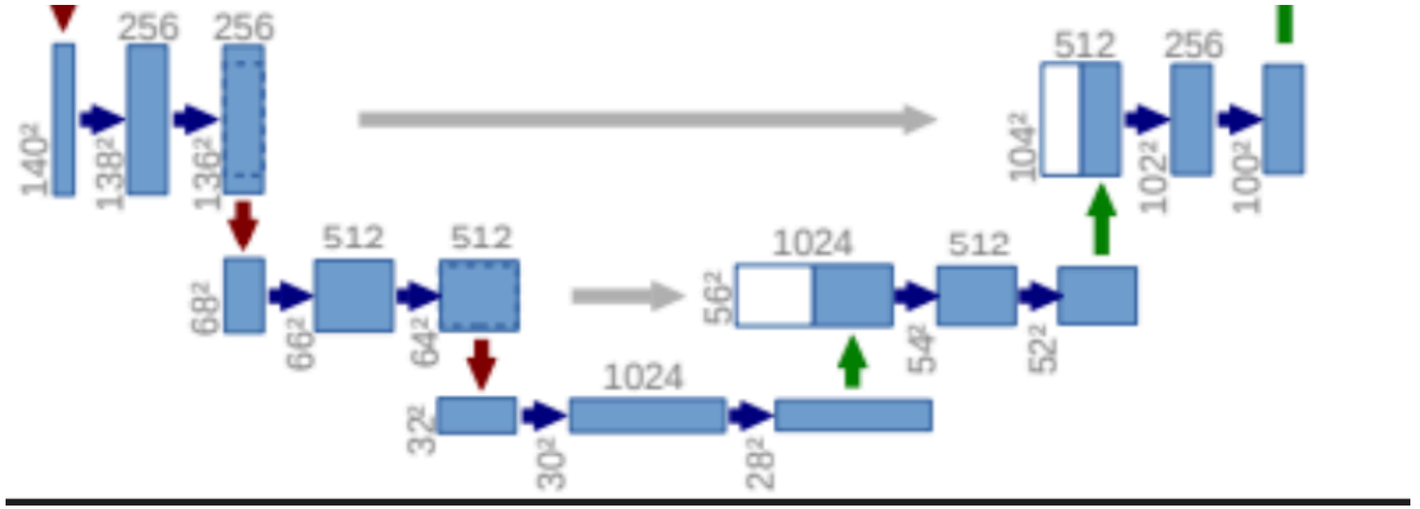
- 입력 이미지의 Localization 파악



U-Net

Skip Architecture

- Encoder의 세부 위치 정보를 Decoder에 전달해 더 정확한 이미지 복원을 돕는 역할



Copy : Encoder의 feature map을 Decoder 쪽으로 복사해서 보냄

Crop : Encoder feature map의 크기를 Decoder feature map과 크기를 맞추기 위해 Encoder feature map의 가장자리 잘라냄 -> padding을 사용하지 않아 잘라내야

Concatenation : feature map을 채널 방향으로 이어 붙이는 것

U-Net 기본 구조 코드

```python
import torch
import torch.nn as nn
import torch.nn.functional as F


class DoubleConv(nn.Module):
    def __init__(self, in_channels, out_channels):
        super(DoubleConv, self).__init__()

        self.conv = nn.Sequential(
            nn.Conv2d(in_channels, out_channels, kernel_size=3, padding=1),  # 3x3 Conv, padding=1로 크기 유지
            nn.ReLU(inplace=True),                                           # 활성함수 ReLU
            nn.Conv2d(out_channels, out_channels, kernel_size=3, padding=1), # 3x3 Conv 한 번 더 적용
            nn.ReLU(inplace=True)                                            # 활성함수 ReLU
        )

    def forward(self, x):
        return self.conv(x)

class UNet(nn.Module):
    def __init__(self, in_channels=3, num_classes=1):
        super(UNet, self).__init__()

        # Encoder: 이미지 크기는 줄이고, 채널 수는 늘림
        self.enc1 = DoubleConv(in_channels, 64)      # 입력 이미지 → 64채널 feature map
        self.pool1 = nn.MaxPool2d(kernel_size=2)     # 크기 절반 감소

        self.enc2 = DoubleConv(64, 128)              # 64채널 → 128채널
        self.pool2 = nn.MaxPool2d(kernel_size=2)     # 크기 절반 감소

        self.enc3 = DoubleConv(128, 256)             # 128채널 → 256채널
        self.pool3 = nn.MaxPool2d(kernel_size=2)     # 크기 절반 감소

        # Bottleneck: 가장 깊은 부분, 고수준 특징 학습
        self.bottleneck = DoubleConv(256, 512)       # 256채널 → 512채널

        # Decoder: 이미지 크기는 다시 키우고, 채널 수는 줄임
        self.up3 = nn.ConvTranspose2d(512, 256, kernel_size=2, stride=2) # 크기 2배 확대
        self.dec3 = DoubleConv(512, 256)             # skip connection으로 256+256=512채널 입력

        self.up2 = nn.ConvTranspose2d(256, 128, kernel_size=2, stride=2) # 크기 2배 확대
        self.dec2 = DoubleConv(256, 128)             # skip connection으로 128+128=256채널 입력

        self.up1 = nn.ConvTranspose2d(128, 64, kernel_size=2, stride=2)  # 크기 2배 확대
        self.dec1 = DoubleConv(128, 64)              # skip connection으로 64+64=128채널 입력

        # Output Layer: 픽셀마다 클래스 예측
        self.out_conv = nn.Conv2d(64, num_classes, kernel_size=1)        # 1x1 Conv로 최종 segmentation map 생성
```

```python
    def forward(self, x):
        # Encoder
        e1 = self.enc1(x)                 # 첫 번째 encoder feature
        p1 = self.pool1(e1)               # downsampling

        e2 = self.enc2(p1)                # 두 번째 encoder feature
        p2 = self.pool2(e2)               # downsampling

        e3 = self.enc3(p2)                # 세 번째 encoder feature
        p3 = self.pool3(e3)               # downsampling

        # Bottleneck
        b = self.bottleneck(p3)           # 가장 깊은 특징 추출

        # Decoder + Skip Connection
        d3 = self.up3(b)                  # upsampling
        d3 = torch.cat([d3, e3], dim=1)   # skip connection: encoder feature와 채널 방향으로 연결
        d3 = self.dec3(d3)                # convolution으로 특징 정리

        d2 = self.up2(d3)                 # upsampling
        d2 = torch.cat([d2, e2], dim=1)   # skip connection
        d2 = self.dec2(d2)                # convolution

        d1 = self.up1(d2)                 # upsampling
        d1 = torch.cat([d1, e1], dim=1)   # skip connection
        d1 = self.dec1(d1)                # convolution

        output = self.out_conv(d1)        # 최종 segmentation map 출력

        return output


model = UNet(in_channels=3, num_classes=1)
print(model)
```

U-Net 학습 방법

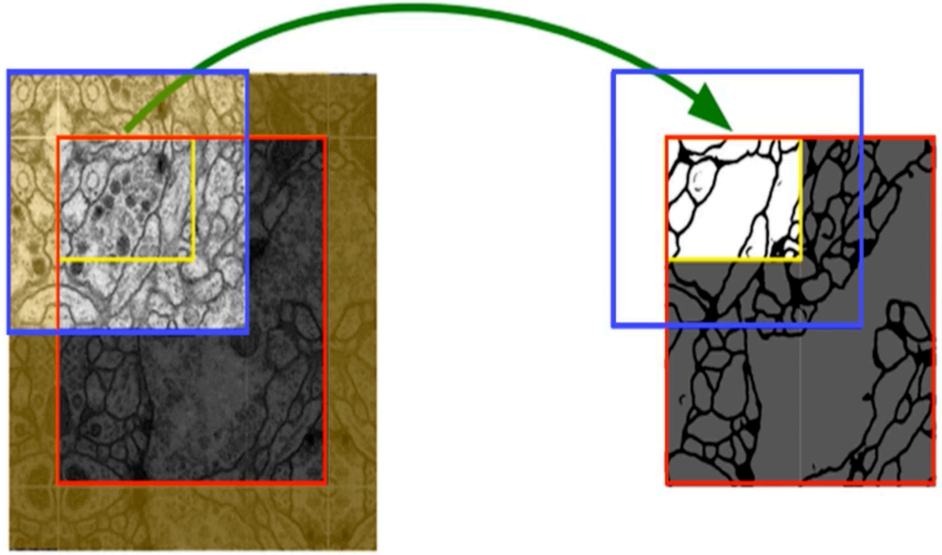
Overlap tile strategy

- 큰 이미지를 작은 tile 단위로 나누어 처리 -> GPU 메모리 부담 줄어듬

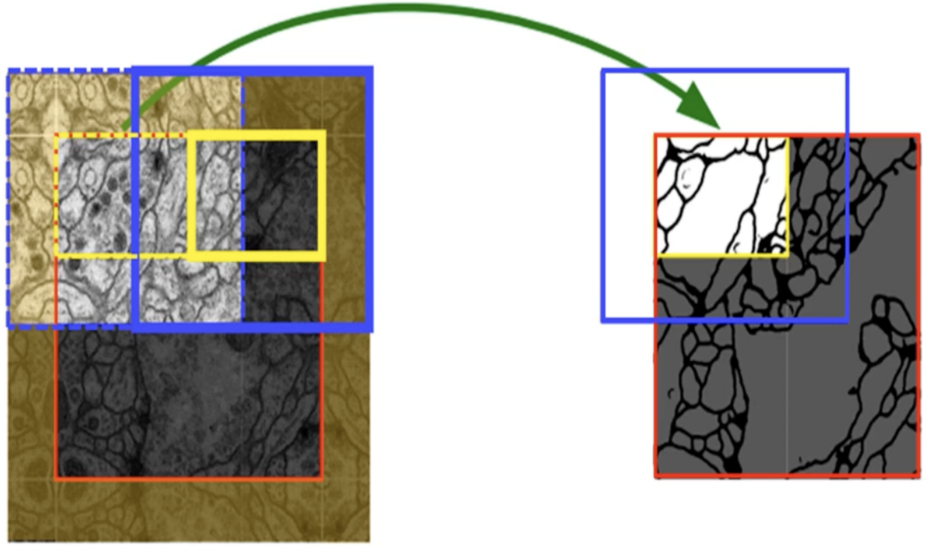
Mirroring : 이미지 가장자리를 거울 처럼 반사시킴

-> 가장자리 바깥쪽을 원본 이미지를 반사한 형태로 채워 경계 부분도 자연 스럽게 예측

ovrelap : tile을 자를 때 서로 일부 영역이 겹치도록 자르는 방식
-> 경계 부분에서도 주변 정보를 충분히 참고



First

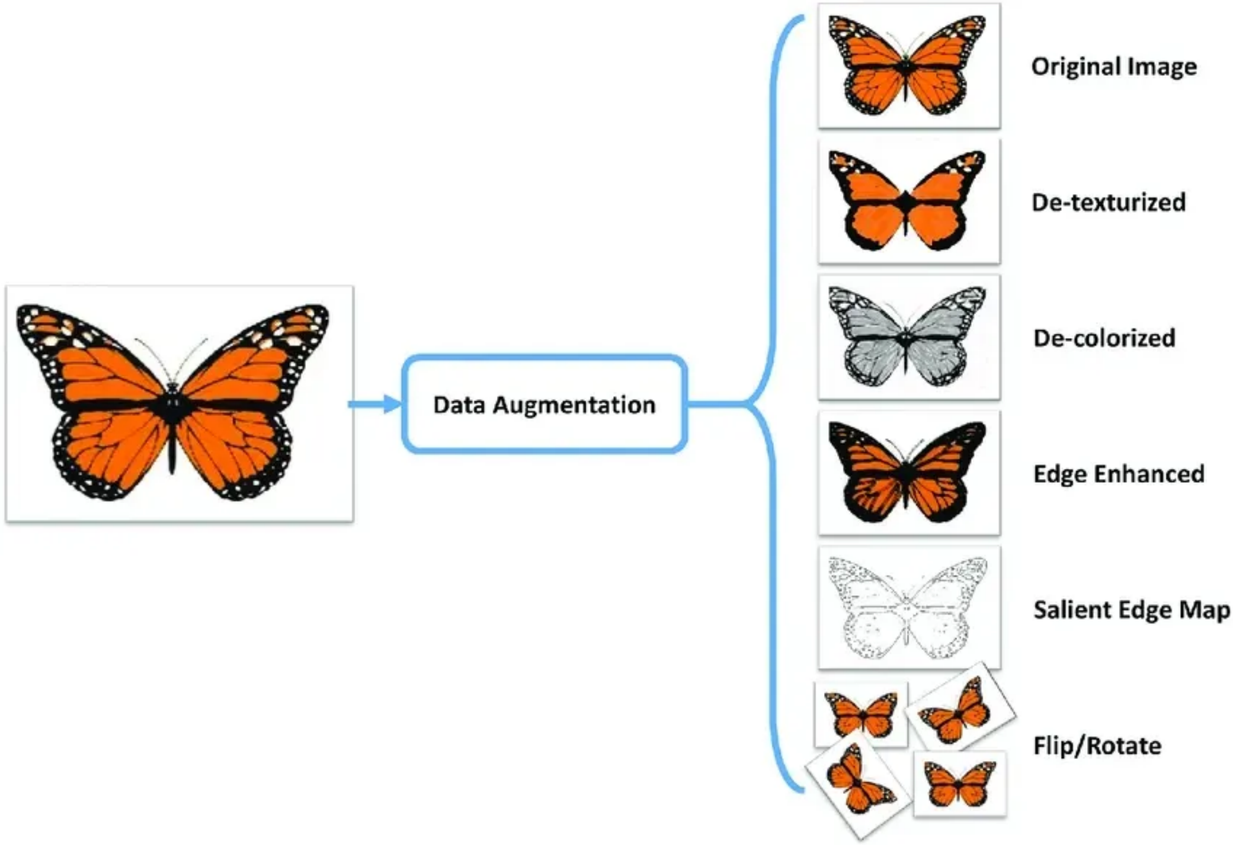


Second

U-Net 학습 방법

Data Augmentation

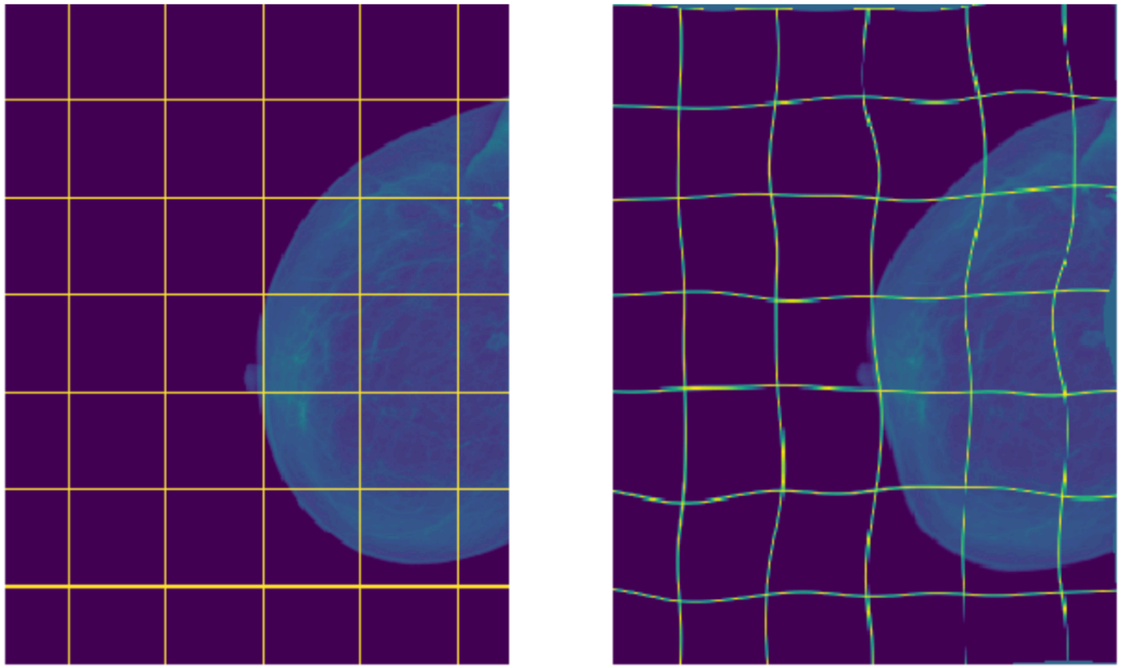
- 원본 데이터에 다양한 변형을 가하여 인공적으로 새로운 학습 데이터를 생성하는 방법
- 특히 의료 영상과 같이 데이터가 적고 수집이 어려운 경우에 중요한 역할



U-Net 학습 방법

Data Augmentation(Elastic Deformation)

- 기본적인 선형 변환에 확률적으로 노이즈를 가해준 것
- 픽셀 별로 이미지가 다른 방향으로 뒤틀리기 때문에 더 현실적인 변화



(a) Original (b) Deformed

U-Net 학습 방법

Training

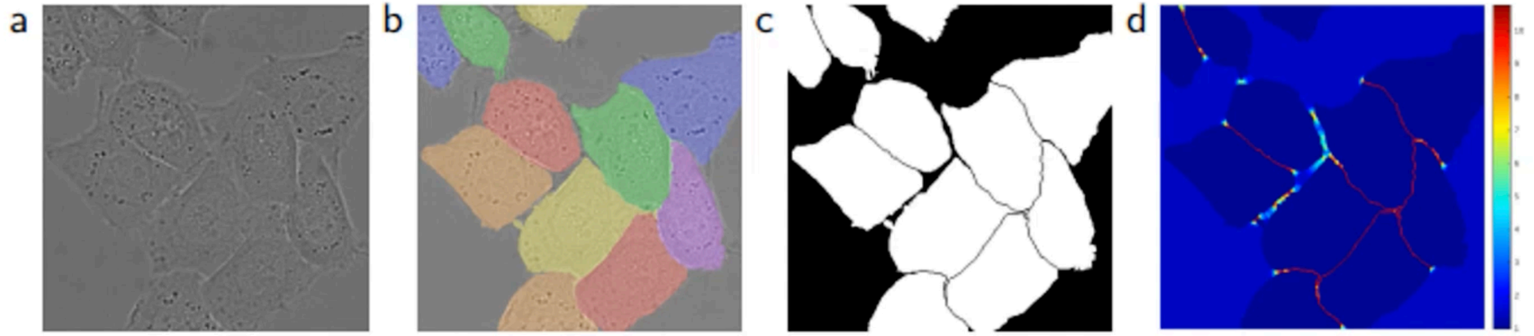


Fig. 3. HeLa cells on glass recorded with DIC (differential interference contrast) microscopy. (**a**) raw image. (**b**) overlay with ground truth segmentation. Different colors indicate different instances of the HeLa cells. (**c**) generated segmentation mask (white: foreground, black: background). (**d**) map with a pixel-wise loss weight to force the network to learn the border pixels.

- 모델의 예측 mask와 정답 mask를 비교하여 loss 계산
  Pixel-wise loss weight map은 세포 경계 부분 오차에 더 큰 가중치 부여 -> 이를 통해 붙어 있는 세포를 더 잘 분리하도록 학습

CNN vs VIT

구분	CNN	VIT
기본 아이디어	필터를 이용해 이미지의 지역적 특징을 추출	이미지를 여러 패치로 나누고 Attention으로 관계 학습
입력 처리 방식	이미지 전체를 픽셀 격자 형태로 처리	이미지를 작은 패치 단위로 나누어 토큰처럼 처리
특징 학습 방식	가까운 영역부터 넓은 영역으로 특징 학습	이미지 패치들 사이의 전체 관계를 한번에 학습
장점	적은 데이터에서도 비교적 잘 학습, 이미지의 지역 특징에 강함	전체 이미지의 문맥과 먼 영역 간 관계 잘 학습함
단점	멀리 떨어진 영역 간 관계를 직접 파악 어려움	많은 데이터와 연산량 필요

감사합니다!